



**Alunos:** Thiago Grandim – Miguel Gonçalves

**Disciplina:** Sistemas Operacionais

**Professor:** Lucas Bragança

**Instituição:** Puc Minas Betim

**Data:** 30/06/26

## **Trabalho Prático 2**

Projetando um Gerenciador de Memória Virtual

### **1. Introdução**

### **2. Descrição do desenvolvimento**

### **3. Resultados obtidos**

### **4. Conclusão**

### **5. Seção específica sobre uso de IA**

### **Repositório no GitHub**

[- Clique aqui para acessar -](#)

## **Introdução**

Este relatório apresenta o desenvolvimento do Trabalho Prático 2 da disciplina de Sistemas Operacionais, que consiste na implementação de um Gerenciador de Memória Virtual (VMM) em linguagem C. O objetivo do projeto é simular, de forma prática, o funcionamento de um sistema de paginação por demanda, compreendendo os passos envolvidos na tradução de endereços lógicos para físicos.

Para isso, o simulador utiliza uma Tabela de Páginas e uma TLB (Translation Lookaside Buffer) para traduzir cada endereço lógico em seu correspondente endereço físico, buscando as páginas ausentes em um arquivo de memória de retaguarda (BACKING\_STORE.bin) sempre que ocorre uma falta de página. Além disso, o sistema implementa uma política de substituição de páginas baseada no algoritmo de

envelhecimento (Aging), responsável por aproximar o comportamento do algoritmo LRU (Least Recently Used) quando todos os quadros de memória física já estão ocupados.

Ao longo deste relatório, são apresentados a arquitetura do simulador, as decisões de projeto adotadas, as dificuldades enfrentadas durante a implementação e os resultados obtidos nos testes realizados, demonstrando o funcionamento correto do gerenciador de memória virtual desenvolvido.

## Descrição do Desenvolvimento

O desenvolvimento partiu da estrutura base fornecida no template do projeto, organizada em módulos separados para tabela de páginas, TLB, memória física e estatísticas, o que facilitou a implementação incremental sugerida no enunciado. Inicialmente, foi implementada a extração do número de página e do deslocamento a partir do endereço lógico, seguida da leitura da tabela de páginas e da busca das páginas no arquivo BACKING\_STORE.bin sempre que ocorria uma falta de página.

Em seguida, foi desenvolvido o gerenciamento da memória física, controlando a ocupação dos quadros disponíveis e tratando corretamente as faltas de página. Para os casos em que a memória física já estava completamente ocupada, foi implementada a política de substituição de páginas baseada no algoritmo de Aging, que aproxima o comportamento do LRU utilizando um contador de envelhecimento e um bit de referência por página. Por último, foi integrada a TLB, com política de substituição FIFO, e implementado o cálculo das estatísticas finais (taxa de page faults e taxa de acerto da TLB).

As principais decisões de projeto seguiram a ordem recomendada no enunciado, priorizando a validação do funcionamento da tradução de endereços e do tratamento de faltas de página antes de adicionar a TLB, o que reduziu a complexidade de depuração. A maior dificuldade encontrada foi manter a consistência entre as três estruturas (TLB, tabela de páginas e memória física) durante a substituição de páginas, já que a remoção de uma página exige a invalidação correta de suas entradas em todas elas. Também houve atenção redobrada na manipulação de bits para extração de página/offset e na ordem das operações do algoritmo de Aging, pontos em que pequenos erros geravam resultados incorretos sem indicar falhas explícitas durante a execução.

## Resultados Obtidos

Foram realizadas duas simulações utilizando arquivos distintos de endereços lógicos “addresses\_random.txt” e “addresses\_location.txt” para validar o funcionamento do gerenciador de memória virtual, incluindo a tradução de endereços, consulta à TLB, tratamento de page faults e substituição de páginas.

## Simulação 1 – addresses\_random.txt

Comando Utilizado: ./vm < data/addresses\_random.txt

```
thiag@minerpc: /mnt/c/Users X + v
Logical address: 6747 Physical address: 3419 Value: 91
Logical address: 43374 Physical address: 29806 Value: 110
Logical address: 2954 Physical address: 17290 Value: -118
Logical address: 29362 Physical address: 25266 Value: -78
Logical address: 37997 Physical address: 10861 Value: 109
Logical address: 32949 Physical address: 11957 Value: -75
Logical address: 51173 Physical address: 19173 Value: -27
Logical address: 46911 Physical address: 7231 Value: 63
Logical address: 8906 Physical address: 17610 Value: -54
Logical address: 20691 Physical address: 3539 Value: -45
Logical address: 37757 Physical address: 13181 Value: 125
Logical address: 9801 Physical address: 17225 Value: 73
Logical address: 31640 Physical address: 26520 Value: -104
Logical address: 40925 Physical address: 27613 Value: -35
Logical address: 52326 Physical address: 18022 Value: 102
Logical address: 7424 Physical address: 12800 Value: 0
Logical address: 14532 Physical address: 25284 Value: -60
Logical address: 61732 Physical address: 29988 Value: 36
Logical address: 34478 Physical address: 17582 Value: -82
Logical address: 3929 Physical address: 3417 Value: 89
Logical address: 4230 Physical address: 17286 Value: -122
Logical address: 44308 Physical address: 28692 Value: 20
Logical address: 55234 Physical address: 30658 Value: -62
Logical address: 34194 Physical address: 7826 Value: -110
Number of Translated Addresses = 10000
Page Faults = 4958
Page Fault Rate = 49.580
TLB Hits = 654
TLB Hit Rate = 6.540
thiag@minerpc: /mnt/c/Users/thiag/Downloads/vm_manager/vm_manager$ |
```

Resultados:

| Métrica                | Valor   |
|------------------------|---------|
| Endereços traduzidos   | 10000   |
| Page Faults            | 4958    |
| Taxa de Page Fault     | 49,580% |
| TLB Hits               | 654     |
| Taxa de Acertos da TLB | 6,540%  |

Análise

Nesta simulação os endereços são distribuídos de forma aleatória, o que reduz significativamente a localidade de referência. Como consequência:

- aproximadamente metade dos acessos gerou page fault;
- houve poucos reaproveitamentos de páginas já carregadas;
- a TLB apresentou baixa eficiência, com apenas 6,54% de acertos;
- o resultado demonstra um cenário de pior caso para o sistema de memória virtual.

## Simulação 2 – addresses\_location.txt

Comando Utilizado: ./vm < data/addresses\_location.txt

```
thiag@minerpc: /mnt/c/Users X + v
Logical address: 41208 Physical address: 9464 Value: -8
Logical address: 40950 Physical address: 30966 Value: -10
Logical address: 41425 Physical address: 6865 Value: -47
Logical address: 25899 Physical address: 22571 Value: 43
Logical address: 41074 Physical address: 9330 Value: 114
Logical address: 40786 Physical address: 30802 Value: 82
Logical address: 40325 Physical address: 25477 Value: -123
Logical address: 26823 Physical address: 26823 Value: -57
Logical address: 40540 Physical address: 23388 Value: 92
Logical address: 45352 Physical address: 15400 Value: 40
Logical address: 40772 Physical address: 30788 Value: 68
Logical address: 40253 Physical address: 25405 Value: 61
Logical address: 41104 Physical address: 9360 Value: -112
Logical address: 41221 Physical address: 6661 Value: 5
Logical address: 41159 Physical address: 9415 Value: -57
Logical address: 40238 Physical address: 25390 Value: 46
Logical address: 41433 Physical address: 6873 Value: -39
Logical address: 40968 Physical address: 9224 Value: 8
Logical address: 40881 Physical address: 30897 Value: -79
Logical address: 40962 Physical address: 9218 Value: 2
Logical address: 40536 Physical address: 23384 Value: 88
Logical address: 40403 Physical address: 25555 Value: -45
Logical address: 40917 Physical address: 30933 Value: -43
Logical address: 40628 Physical address: 23476 Value: -76
Logical address: 41151 Physical address: 9407 Value: -65
Logical address: 41364 Physical address: 6804 Value: -108
Logical address: 41374 Physical address: 6814 Value: -98
Logical address: 40344 Physical address: 25496 Value: -104
Logical address: 40303 Physical address: 25455 Value: 111
Logical address: 20976 Physical address: 22768 Value: -16
Logical address: 40303 Physical address: 25455 Value: 111
Logical address: 16797 Physical address: 15005 Value: -99
Logical address: 23011 Physical address: 26851 Value: -29
Logical address: 41065 Physical address: 9321 Value: 105
Logical address: 42561 Physical address: 19265 Value: 65
Logical address: 30126 Physical address: 3246 Value: -82
Logical address: 40251 Physical address: 25403 Value: 59
Logical address: 41293 Physical address: 6733 Value: 77
Logical address: 40449 Physical address: 23297 Value: 1
Logical address: 56232 Physical address: 32168 Value: -88
Logical address: 41031 Physical address: 9287 Value: 71
Logical address: 40363 Physical address: 25515 Value: -85
Logical address: 40244 Physical address: 25396 Value: 52
Logical address: 40488 Physical address: 23336 Value: 40
Logical address: 40975 Physical address: 9231 Value: 15
Number of Translated Addresses = 10000
Page Faults = 1164
Page Fault Rate = 11.640
TLB Hits = 7925
TLB Hit Rate = 79.250
thiag@minerpc: /mnt/c/Users/thiag/Downloads/vm_manager/vm_manager$ |
```

### Resultados

| Métrica                | Valor   |
|------------------------|---------|
| Endereços traduzidos   | 10000   |
| Page Faults            | 1164    |
| Taxa de Page Fault     | 11,640% |
| TLB Hits               | 7925    |
| Taxa de Acertos da TLB | 79,250% |

## Análise

Neste conjunto de testes existe forte **localidade de referência**, ou seja, os acessos tendem a reutilizar páginas recentemente utilizadas.

Como resultado:

- a quantidade de page faults caiu para apenas 11,64%;
- a maior parte das traduções foi encontrada diretamente na TLB;
- a taxa de acertos da TLB chegou a 79,25%, indicando excelente aproveitamento do cache de traduções;
- esse comportamento representa um cenário mais próximo do observado em aplicações reais.

## Validação funcional

Durante os testes foi possível verificar que:

- tradução correta dos endereços lógicos para físicos;
- leitura correta dos valores armazenados na memória física;
- tratamento adequado de page faults, carregando páginas do arquivo de respaldo (BACKING\_STORE.bin);
- atualização da tabela de páginas;
- atualização da TLB após cada tradução;
- contabilização correta das estatísticas de execução (page faults e TLB hits).

## Comparação dos resultados

| Métrica                | Random | Locality |
|------------------------|--------|----------|
| Endereços traduzidos   | 10000  | 10000    |
| Page Faults            | 4958   | 1164     |
| Taxa de Page Fault     | 49,58% | 11,64%   |
| TLB Hits               | 654    | 7925     |
| Taxa de Acertos da TLB | 6,54%  | 79,25%   |

## Conclusão

O desenvolvimento deste trabalho permitiu compreender, na prática, os mecanismos envolvidos na gerência de memória virtual em sistemas operacionais, especialmente a tradução de endereços lógicos para físicos por meio de tabela de páginas e TLB, além do tratamento de faltas de página com paginação por demanda. A implementação do algoritmo de Aging como aproximação do LRU evidenciou como sistemas reais conseguem aproximar políticas de substituição sofisticadas sem o custo de armazenar históricos completos de acesso.

Os testes realizados com os arquivos de endereços fornecidos confirmaram o funcionamento correto do simulador, com taxas de page faults e de acerto da TLB condizentes com o esperado para os padrões de acesso testados. De forma geral, o projeto cumpriu seu objetivo de simular, de maneira didática, os principais desafios enfrentados na gerência de memória virtual, reforçando o entendimento teórico da disciplina por meio de uma implementação prática completa.

## Uso de IA para o desenvolvimento

Anexamos o template do projeto, juntamente com suas especificações em PDF e explicamos um pouco pedindo para o CLAUDE, (IA que foi utilizada) para compreender o entendimento dela sobre o projeto, utilizamos a IA por etapas para evitar erros ou cortes indevidos em partes desejadas, dessa maneira começamos apenas buscando entender o projeto.

Estou desenvolvendo um projeto acadêmico de Sistemas Operacionais chamado Virtual Memory Manager.  
Vou fornecer um projeto-base completo (vm\_manager.zip) e a especificação oficial do trabalho (PDF).  
Antes de implementar qualquer código, quero que você faça apenas uma análise completa do projeto.  
Objetivos desta etapa:

- Ler toda a especificação.
- Ler toda a estrutura do projeto-base.
- Identificar todos os arquivos existentes.
- Explicar a responsabilidade de cada arquivo.
- Localizar todas as funções marcadas com TODO.
- Explicar exatamente o que cada TODO deve fazer segundo a especificação.
- Explicar como os módulos deverão interagir entre si.
- Criar um plano completo de implementação seguindo exatamente a ordem sugerida pelo professor.
- Não implemente nenhum código nesta etapa.
- Não altere a arquitetura do projeto-base.
- Não crie novos arquivos.
- Não remova arquivos existentes.
- Apenas analise toda a estrutura e apresente um plano técnico detalhado de implementação."

Mostrar menos

Os prompts utilizados foram organizados nas seguintes etapas:

## Etapa 1 – Extração de Página e Offset

Seu entendimento foi perfeito aqui vai algumas Regras obrigatórias:

- Não altere a arquitetura existente.
  - Não modifique nomes de funções.
  - Não altere assinaturas dos métodos.
  - Não altere arquivos que não contenham TODOS relacionados à etapa atual.
  - Não substitua estruturas existentes por outras mais sofisticadas.
  - Não reorganize o projeto.
  - Preserve comentários existentes.
  - Implemente apenas o mínimo necessário para cumprir a especificação.
  - O código deve parecer produzido por um estudante de graduação, priorizando clareza e aderência ao template fornecido.
- Implementação da Etapa 1 (Extração de Página e Offset)
- Você já analisou completamente o projeto-base e a especificação do trabalho.
- Agora vamos iniciar a implementação seguindo exatamente a ordem recomendada pelo professor.
- Objetivo desta etapa
- Implementar apenas a extração correta do número da página e do deslocamento (offset) a partir dos endereços lógicos recebidos pelo programa.
- A implementação deve seguir rigorosamente a especificação do trabalho:
- O endereço lógico recebido possui 32 bits.
  - Apenas os 16 bits menos significativos devem ser considerados.
  - Os 16 bits devem ser separados em:
    - 8 bits para o número da página.
    - 8 bits para o deslocamento (offset).
  - Utilize máscaras de bits conforme especificado no enunciado.
- O resultado esperado é equivalente à lógica:
- mascarar os 16 bits inferiores;
  - extrair a página;
  - extrair o offset.
- O que espero na resposta
- Explique rapidamente qual trecho será implementado.
  - Mostre somente o código que foi alterado.
  - Explique linha por linha o raciocínio da implementação.
  - Confirme que nenhuma outra funcionalidade do projeto foi modificada.
  - Informe como validar manualmente essa etapa antes de prosseguirmos.

Mostrar menos

Objetivo solicitado à IA:

Implementar a extração do número da página e do deslocamento (offset) a partir dos 16 bits menos significativos do endereço lógico, utilizando operações de máscara de bits.

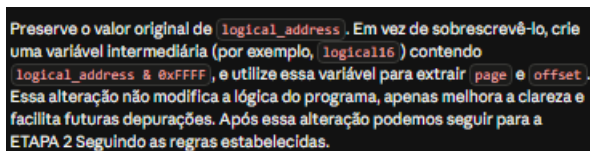
Como a IA foi utilizada:

Geração de código;

Explicação linha a linha da implementação;

Explicação sobre operações com máscaras de bits;

Orientação para validação manual da etapa.



Preserve o valor original de `logical_address`. Em vez de sobrescrevê-lo, crie uma variável intermediária (por exemplo, `logical16`) contendo `logical_address & 0xFFFF`, e utilize essa variável para extrair `page` e `offset`. Essa alteração não modifica a lógica do programa, apenas melhora a clareza e facilita futuras depurações. Após essa alteração podemos seguir para a ETAPA 2 Seguindo as regras estabelecidas.

Posteriormente, foi solicitado um pequeno ajuste para preservar o valor original da variável `logical_address`, utilizando uma variável intermediária (`logical16`), visando melhorar a clareza do código e facilitar futuras depurações.

## Etapa 2 – Implementação da Tabela de Páginas



#### Implementação da Tabela de Páginas (Etapa Básica)

Você já analisou completamente o projeto-base e a especificação do trabalho. Agora vamos implementar apenas a funcionalidade básica da Tabela de Páginas.

##### Regras obrigatórias

- Não altere a arquitetura existente do projeto.
  - Não crie novos arquivos.
  - Não remova arquivos.
  - Não altere nomes de funções.
  - Não altere assinaturas das funções.
  - Não reorganize o projeto.
  - Preserve todos os comentários existentes.
  - Não implemente funcionalidades pertencentes às próximas etapas.
  - Não faça refatorações.
  - Não mova código entre arquivos.
  - Utilize apenas as estruturas já existentes no projeto-base.
  - O código deve ser simples, legível e compatível com o estilo esperado de um projeto acadêmico de graduação.
- Caso encontre algum problema fora do escopo desta etapa, apenas informe ao final da resposta. Não implemente nenhuma solução adicional.

##### Objetivo desta etapa

Implementar somente a funcionalidade básica da Tabela de Páginas, permitindo que ela armazene e consulte o mapeamento entre páginas virtuais e quadros físicos.

Nesta etapa devem ser implementadas apenas as seguintes funções em `src/page_table.c`:

- `page_table_lookup()`
  - `page_table_update()`
  - `page_table_invalidate()`
  - `page_table_set_reference()`
- Não implementar nesta etapa
- Mesmo que existam TODOS no mesmo arquivo, não implemente:
- `page_table_update_aging()`
- Essa função pertence ao algoritmo Aging (LRU aproximado) e será implementada posteriormente, durante a etapa de substituição de páginas.
- Também não implemente qualquer lógica relacionada a:
- seleção de página vítima;
  - substituição de páginas;
  - TLB;
  - estatísticas;
  - leitura do BACKING\_STORE;
  - gerenciamento de quadros;
  - alterações em `memory.c`;
  - alterações em `main.c`.
- Requisitos funcionais
- A implementação deve permitir que:
- `page_table_lookup(page)`
- Verifique se a página está válida.
- Caso esteja válida, retorne o frame correspondente.
  - Caso contrário, retorne o valor indicado pelo template (normalmente `-1`).
- `page_table_update(page, frame)`
- Atualize corretamente a entrada da tabela:
- frame correspondente;

- página válida;
  - Inicialize o bit de referência conforme esperado pelo projeto;
  - Inicialize o contador de aging em um estado coerente para uma página recém-carregada.
- Não realize nenhuma lógica de substituição.

`page_table_invalidate(page)`

Ao invalidar uma página:

- marcar a entrada como inválida;
- limpar o frame associado (caso o template faça isso);
- zerar bit de referência;
- zerar contador de aging.

`page_table_set_reference(page)`

Quando uma página for acessada:

- marcar apenas o bit de referência. Nenhuma outra informação deve ser alterada. Importante Não modifique a estrutura interna das entradas da tabela de páginas. Não altere tipos. Não altere arrays. Não altere constantes. Não acrescente novos campos. Utilize exatamente as estruturas fornecidas pelo projeto-base. O que espero na resposta
- Organize sua resposta da seguinte maneira:

- **Resumo da Implementação** Explique brevemente o objetivo desta etapa.
- **Código alterado** Mostre apenas os trechos efetivamente modificados.
- Não envie arquivos completos.

- **Explicação detalhada**

Explique função por função:

- o que ela faz;
  - por que foi implementada dessa maneira;
  - como ela será utilizada nas próximas etapas.
  - **Confirmação de escopo**
- Confirme explicitamente que:
- nenhum outro arquivo foi modificado;
  - nenhuma funcionalidade futura foi implementada;
  - `page_table_update_aging()` permaneceu inalterada.
- **Validação**

Explique como validar manualmente esta etapa antes de prosseguirmos. A validação deve considerar apenas a funcionalidade da Tabela de Páginas, sem depender da implementação do TLB ou do algoritmo de substituição.

Objetivo final desta etapa

Ao término desta implementação, a Tabela de Páginas deverá estar completamente funcional para:

- registrar páginas carregadas;
- consultar páginas existentes;
- invalidar páginas;
- registrar acessos através do bit de referência. Sem ainda implementar o algoritmo Aging nem qualquer política de substituição.

Objetivo solicitado à IA:

Implementar apenas as funções básicas da Tabela de Páginas:

`page_table_lookup()`

`page_table_update()`

`page_table_invalidate()`

`page_table_set_reference()`

Sem implementar ainda o algoritmo Aging, TLB ou substituição de páginas.

Como a IA foi utilizada:

Geração de código;

Explicação do funcionamento de cada função;

Esclarecimento da interação entre as estruturas da tabela de páginas;

Auxílio na validação da implementação.

Etapa 3 – Memória Física e Tratamento Básico de Page Fault

### Memória Física e Tratamento Básico de Page Fault

Você já analisou completamente o projeto-base e a especificação do trabalho. As etapas anteriores já foram concluídas:

- Extração de página e offset em `main.c`
- Implementação básica da tabela de páginas em `page_table.c`

Agora vamos implementar a próxima etapa do simulador: gerenciamento básico da memória física.

Objetivo desta etapa

Implementar o funcionamento básico da memória física para permitir:

- Carregar páginas do `BACKING_STORE.bin`.
- Associar páginas a quadros físicos livres.
- Registrar corretamente o mapeamento em `frame_to_page`.
- Ler bytes armazenados na memória física. Arquivo permitido Modificar apenas:

`src/memory.c`

Funções que devem ser implementadas nesta etapa

1. Completar `handle_page_fault()`

Assuma que ainda existem quadros livres disponíveis.

Implementar:

- Atualização de `frame_to_page[frame]`.
- Atualização da tabela de páginas usando `page_table_update(page, frame)`.
- Utilização do frame retornado por `find_free_frame()`.
- Retorno correto do frame carregado. A leitura do `BACKING_STORE.bin` já foi implementada anteriormente e deve ser preservada.
- Implementar `read_memory()`

Esta função deve:

- Receber `frame` e `offset`.
- Retornar o byte armazenado em:

`physical_memory[frame][offset]`

Sem lógica adicional.

Não implementar nesta etapa

Mesmo que existam TODOS relacionados em `memory.c`, NÃO Implementar:

Substituição de páginas

Não implementar:

- `select_victim_page()`
- escolha de página vítima
- reutilização de quadros ocupados
- invalidação de páginas
- Aging (LRU aproximado)

Não implementar:

- `page_table_update_aging()`
- contadores de envelhecimento
- atualização de bits de referência

Integração com TLB

Integração com TLB

Não implementar:

- `tlb_remove()`

lógica de substituição do TLB

qualquer alteração em `tlb.c`

Estatísticas

Não implementar:

- contadores
- métricas
- taxas

Importante

Nesta etapa, quando houver frame livre:

- A página deve ser carregada do `BACKING_STORE.bin`.
- O frame escolhido deve ser registrado em `frame_to_page`.
- A tabela de páginas deve ser atualizada.
- O frame deve ser retornado. Caso o código possua um bloco referente a

`if (frame == -1)`

mantenha a estrutura atual exatamente como está.

Não implemente ainda nenhuma lógica de substituição.

O que espero na resposta

Organize a resposta da seguinte forma:

- Resumo da implementação
- Código alterado
- Explicação detalhada

Mostre apenas os trechos modificados.

Não envie arquivos completos.

Explique:

- como a página é carregada;
- como o frame é registrado;
- como a tabela de páginas é atualizada;
- como `read_memory()` funciona;
- como essas alterações interagem com os módulos já implementados (`main.c` e `page_table.c`).

Confirmação de escopo

Confirme explicitamente que:

- nenhum outro arquivo foi modificado;
- nenhuma funcionalidade futura foi implementada;
- não houve alteração no algoritmo de substituição;
- `select_victim_page()` permaneceu inalterada.

Validação

Explique como validar esta etapa utilizando:

- `BACKING_STORE.bin`
- tabela de páginas
- leitura de memória física

sem depender ainda do TLB ou do algoritmo Aging.

Objetivo final desta etapa

Ao final desta implementação, o simulador deverá ser capaz de:

- carregar páginas do disco para a memória física;
- registrar corretamente os mapeamentos página → frame;
- consultar a tabela de páginas;
- acessar bytes armazenados na memória física; ainda sem implementar substituição de páginas.

Objetivo solicitado à IA:

Implementar o gerenciamento básico da memória física para permitir:

carregamento de páginas do BACKING\_STORE.bin;

associação entre páginas e quadros físicos;

atualização da tabela de páginas;

implementação da função read\_memory().

Sem implementar ainda qualquer política de substituição de páginas.

Como a IA foi utilizada:

Geração de código;

Explicação do fluxo de tratamento de Page Fault;

Explicação da interação entre memória física e tabela de páginas;

Auxílio na validação da implementação.

Etapa 4 – Integração do Fluxo Principal

Integração do Fluxo Principal (sem Aging)

Analisai o andamento da implementação e decidi alterar um pouco a ordem das próximas etapas.

Inicialmente íamos implementar o algoritmo Aging (LRU aproximado), porém prefiro seguir exatamente a recomendação do professor descrita no PDF. O próprio enunciado recomenda que a tradução de endereços esteja completamente funcional antes da integração das próximas funcionalidades, como TLB e política de substituição de páginas.

Por isso, não quero implementar o Aging nesta etapa.

Quero primeiro deixar todo o fluxo básico do simulador funcionando corretamente utilizando apenas:

- Extração de página e offset (já implementada);
- Tabela de páginas (já implementada);
- Memória física (já implementada);
- Tratamento básico de Page Fault (já implementado para frames livres). Somente depois iremos implementar:
- Algoritmo Aging (LRU aproximado);
- Política de substituição de páginas;
- TLB;
- Estatísticas. Objetivo desta etapa Quero integrar completamente o fluxo principal do simulador em `src/main.c`, considerando apenas o cenário em que ainda existem quadros livres na memória física. O objetivo é deixar o simulador capaz de traduzir corretamente endereços lógicos para físicos, sem utilizar ainda o algoritmo de substituição. Arquivo permitido Modificar apenas:

src/main.c

Implementar nesta etapa

Implemente apenas os TODOS necessários para que o fluxo principal funcione corretamente utilizando os módulos já implementados.

O fluxo esperado é:

- Receber um endereço lógico.
- Mascarar os 16 bits inferiores (já implementado).
- Extrair página e offset (já implementado).
- Consultar o TLB utilizando a função existente.
  - Nesta etapa ele continuará retornando miss, pois ainda não foi implementado.
- Consultar a Tabela de Páginas.
- Caso a página não esteja carregada:
  - chamar `handle_page_fault(page)`;
- Após obter o frame:
  - marcar o bit de referência usando `page_table_set_reference(page)`;
- Calcular corretamente o endereço físico.
- Ler o byte utilizando `read_memory(frame, offset)`.
- Permitir que a saída do programa utilize os valores calculados corretamente.

NÃO implementar nesta etapa

Não implementar nenhuma funcionalidade relacionada a:

Algoritmo Aging

Não implementar:

11. `page table update aging()`
12. atualização dos contadores
13. atualização dos bits de referência
14. seleção por LRU aproximado
- Substituição de páginas
- Não implementar:
15. `select_victim_page()`
16. reutilização de quadros ocupados
17. invalidação de páginas
18. remoção do TLB
- TLB
- Não implementar:
19. `tlb_lookup()`
20. `tlb_insert()`
21. `tlb_remove()`
- Apenas utilize as funções já existentes.
- Estatísticas
- Não implementar:
22. `count_address()`
23. `count_page_fault()`
24. `count_tlb_hit()`
25. `print_statistics()` Muito importante Se durante esta implementação surgir algum trecho do `main.c` que dependa diretamente do Aging ou da substituição de páginas, preserve exatamente como está e informe isso ao final da resposta. Não implemente soluções provisórias. O que espero na resposta Organize a resposta da seguinte forma.
26. Resumo da implementação Explique o objetivo desta etapa.
27. Código alterado Mostre apenas os trechos modificados. Não envie arquivos completos.
28. Explicação detalhada
- Explique:
29. como o fluxo principal agora funciona;
30. como ocorre a consulta da tabela de páginas;
31. quando ocorre um Page Fault;
32. como o endereço físico é calculado;
33. como `read_memory()` é utilizado;
34. como essa implementação interage com os módulos desenvolvidos anteriormente.
35. Confirmação de escopo
- Confirme explicitamente que:
36. apenas `src/main.c` foi alterado;
37. nenhuma funcionalidade de Aging foi implementada;
38. nenhuma política de substituição foi implementada;
39. nenhum código do TLB foi implementado;
40. nenhuma estatística foi implementada.
41. Validação Explique como validar esta etapa utilizando os arquivos de teste fornecidos pelo projeto, considerando apenas o cenário em que a memória física ainda possui quadros livres. Informe também quais limitações ainda existirão antes da implementação do Aging e do TLB.

Objetivo final Ao final desta etapa, quero que o simulador esteja funcional para o fluxo básico de tradução de endereços, conforme a recomendação do professor, deixando apenas as funcionalidades avançadas (Aging, substituição de páginas, TLB e estatísticas) para as próximas etapas.

Mostrar menos

Objetivo solicitado à IA:

Integrar o fluxo principal do simulador em main.c, realizando:

tradução de endereços lógicos;

consulta à tabela de páginas;

tratamento de Page Fault;

cálculo do endereço físico;

leitura da memória física.

Sem implementar ainda o algoritmo Aging, TLB, estatísticas ou substituição de páginas.

Como a IA foi utilizada:

Geração de código;

Explicação do fluxo completo de tradução de endereços;

Esclarecimento sobre a integração entre os módulos já implementados;

Auxílio na validação do funcionamento.

Etapa 5 – Algoritmo Aging e Política de Substituição de Páginas

## Implementação Completa do Algoritmo Aging e Política de Substituição de Páginas

Após revisar o andamento do projeto, decidi que agora é o momento de implementar completamente a política de substituição de páginas.

Até este ponto já concluímos:

- Extração de página e offset;
- Tabela de páginas (funcionalidade básica);
- Leitura do BACKING\_STORE;
- Gerenciamento básico da memória física;
- Fluxo principal de tradução de endereços.

Tudo isso foi implementado seguindo a recomendação do professor presente no PDF, deixando a tradução básica completamente funcional antes da política de substituição.

Agora quero implementar todo o módulo responsável pelo gerenciamento da memória chela, utilizando o algoritmo Aging (LRU aproximado) exigido pelo trabalho.

Regras obrigatórias

- Não altere a arquitetura existente.
  - Não crie novos arquivos.
  - Não remova arquivos.
  - Não altere nomes de funções.
  - Não altere assinaturas.
  - Não reorganize o projeto.
  - Preserve todos os comentários existentes.
  - Não faça refatorações.
  - Não mova código entre arquivos.
  - Utilize apenas as estruturas fornecidas pelo projeto-base.
  - Mantenha o código simples, organizado e compatível com o estilo esperado de um projeto acadêmico. Caso encontre algum problema fora do escopo desta etapa, apenas relate ao final da resposta. Objetivo desta etapa Implementar completamente o gerenciamento da memória física quando não existirem mais quadros livres. Ao final desta etapa, o simulador deverá ser capaz de substituir corretamente páginas utilizando o algoritmo Aging (LRU aproximado). Arquivos permitidos
- Modificar apenas:

```
src/page_table.c
src/memory.c
```

Nenhum outro arquivo deve ser alterado.

Implementar em `page_table.c`

`page_table_update_aging()`

Implementar exatamente conforme descrito pelo professor.

Para todas as páginas válidas:

1. Deslocar o contador de aging um bit para a direita.
  2. Se o bit de referência estiver marcado:
    - Inserir 1 no bit mais significativo ( `0x80` ).
  3. Limpar o bit de referência.
  4. Repetir para todas as páginas válidas.
- Não altere nenhuma outra função deste arquivo.

Implementar em `memory.c`

4. Repetir para todas as páginas válidas.

Não altere nenhuma outra função deste arquivo.

Implementar em `memory.c`

`select_victim_page()`

Implementar a seleção da página vítima utilizando o algoritmo Aging.

Requisitos:

5. Percorrer todas as páginas válidas.
  6. Comparar seus contadores de aging.
  7. Selecionar a página que possuir o menor contador.
  8. Em caso de empate, manter um critério consistente (por exemplo, a primeira encontrada).
- Não utilizar nenhuma lógica diferente da especificada pelo trabalho.
- Completar `handle_page_fault()`
- Quando não existir frame livre ( `frame == -1` ):
- Implementar completamente o fluxo de substituição.

A sequência deve ser:

9. Escolher a página vítima utilizando `select_victim_page()`.
  10. Descobrir em qual frame essa página está utilizando as funções já existentes da tabela de páginas.
  11. Invalidar a página vítima utilizando `page_table_invalidate()`.
  12. Remover a entrada correspondente do TLB utilizando `tlb_remove()`.
- Importante:
- Mesmo que o TLB ainda não esteja implementado, utilize a função existente exatamente como previsto na arquitetura do projeto. Ela será implementada posteriormente.

Depois disso:

13. Utilizar o frame liberado.
14. Carregar a nova página do BACKING\_STORE.
15. Atualizar `frame_to_page`.
16. Atualizar a tabela de páginas utilizando `page_table_update()`.
17. Retornar o frame utilizado.

NÃO implementar nesta etapa

Não implementar:

TLB

Não alterar:

18. `tlb_lookup()`
19. `tlb_insert()`
20. `tlb_remove()`

Apenas utilizar `tlb_remove()` onde necessário.

Estatísticas

Não alterar:

21. `count_address()`
  22. `count_page_fault()`
  23. `count_tlb_hit()`
  24. `print_statistics()`
- Fluxo principal Não alterar:

Muito importante

Quero que esta implementação siga exatamente a arquitetura prevista pelo projeto-base.

Não quero otimizações.

Não quero melhorias.

Não quero validações extras.

Não quero alterações de estilo.



Apenas implementar exatamente o comportamento esperado pelo professor.  
O que espero na resposta  
Organize a resposta da seguinte forma.

1. Resumo da implementação  
Explique rapidamente o objetivo desta etapa.
2. Código alterado  
Mostre apenas os trechos modificados.  
Não envie arquivos completos.
3. Explicação detalhada  
Explique separadamente:  
`page_table_update_aging()`
  - como funciona o contador de aging;
  - como o bit de referência é utilizado;
  - por que esse algoritmo aproxima o LRU.`select_victim_page()`  
Explique:
  - como a vítima é escolhida;
  - por que o menor contador representa a melhor candidata à substituição.`handle_page_fault()`  
Explique passo a passo:
  - escolha da vítima;
  - recuperação do frame;
  - invalidação da tabela de páginas;
  - remoção do TLB;
  - carregamento da nova página;
  - atualização do frame;
  - atualização da tabela. Explique também como essas alterações se integram ao fluxo que já foi implementado nas etapas anteriores.

Confirmação de escopo  
Confirme explicitamente que:

- apenas `src/page_table.c` e `src/memory.c` foram modificados;
- nenhum outro arquivo foi alterado;
- nenhuma implementação de TLB foi realizada;
- nenhuma estatística foi implementada;
- `main.c` permaneceu inalterado.

Validação  
Explique como validar esta etapa utilizando os arquivos fornecidos pelo projeto.  
Quero saber:

- como verificar que a substituição está funcionando;
- como confirmar que o algoritmo Aging está escolhendo corretamente as vítimas;
- quais diferenças devem ser observadas entre `addresses_random.txt` e `addresses_location.txt`.

Objetivo final desta etapa  
Ao término desta implementação, o simulador deverá ser capaz de:

- carregar páginas normalmente enquanto houver frames livres;
- substituir páginas corretamente quando a memória física estiver cheia;
- utilizar o algoritmo Aging (LRU aproximado) conforme especificado pelo professor;
- manter a tabela de páginas consistente após cada substituição;
- manter a arquitetura preparada para a implementação do TLB e das estatísticas nas próximas etapas.

Objetivo solicitado à IA:

Implementar:

`page_table_update_aging();`

`select_victim_page();`

complementação de `handle_page_fault()` para memória cheia, utilizando o algoritmo Aging (aproximação do LRU).

Como a IA foi utilizada:



Geração de código;

Explicação detalhada do algoritmo Aging;

Explicação do processo de seleção da página vítima;

Explicação da integração com o restante do simulador;

Orientação para testes e validação da política de substituição.

Forma de utilização da IA

Durante essas etapas, a Inteligência Artificial foi utilizada como ferramenta de apoio ao desenvolvimento, principalmente para:

geração de trechos de código compatíveis com a arquitetura fornecida;

explicação de conceitos relacionados à memória virtual;

esclarecimento do funcionamento das funções implementadas;

auxílio na depuração de pequenos problemas;

orientação sobre formas de testar e validar cada etapa do projeto.

A IA não foi utilizada para desenvolver o projeto completo de forma automática, mas sim como um recurso de apoio semelhante à consulta de documentação técnica, auxiliando na compreensão da especificação e na implementação gradual das funcionalidades.

## Continuidade do desenvolvimento

Após essas etapas iniciais, o restante do projeto foi desenvolvido sem o uso de Inteligência Artificial. Isso ocorreu porque o limite de tokens disponível na ferramenta Claude foi atingido durante o desenvolvimento, impossibilitando sua continuidade como ferramenta de apoio.

Dessa forma, as funcionalidades restantes, incluindo ajustes, integrações finais, correções e testes complementares, foram implementadas manualmente, com base na especificação do trabalho, na documentação da disciplina e nos conhecimentos adquiridos ao longo do desenvolvimento do projeto.